
OpenSDK Development Guide

Revision History

No.	Revision Content	Revised by	Revision Time	Version	Approved by
1	Optimized the document	Bo Tao	March 10, 2025	1.1.2	
2	Upgrade SDK version, documentation remains unchanged.	Bo Tao	June 19, 2025	1.2.0	
3	Add functionality for downloading device recordings.	Bo Tao	November 18, 2025	1.3.0	
4	Add ARC device talk	Bo Tao	Jan 27, 2026	1.4.0	
5	Add x86 version	Bo Tao	Apr 02, 2026	1.5.0	
6	Add p2p and cloudRecord	Bo Tao	Apr 13, 2026	3.2.0	
7	Adapt to low-power devices	Bo Tao	May 28, 2026	3.3.0	
8	Resolve the P2P key truncation issue	Bo Tao	Jul 03, 2026	3.3.1	

Table of Contents

OpenSDK Development Guide.....	1
Table of Contents.....	2
1 Overview	7
2 Configuration of Development Environment.....	8
2.1 SDK Product.....	8
2.2 Adding SDK Package	9
2.3 Visual Studio Project Configuration	9
3 SDK Interface Description.....	10
3.1 Overview	10
3.2 LCOpenSDK_Define	10
3.2.1 Parameter Enumeration	10
3.2.1.1 E_STREAM_TYPE	10
3.2.1.2 E_MEDIA_FORMAT	11
3.2.1.3 E_MEDIA_TYPE	11
3.2.1.4 E_LOG_LEVEL	11
3.2.1.5 E_DEVICE_STATE	12
3.2.2 Callback Enumeration.....	12
3.2.2.1 E_RESULT_PROTO_TYPE	12
3.2.2.2 E_RESULT_PROTO_TYPE_RTSP_STATE.....	13
3.2.2.3 E_RESULT_PROTO_TYPE_DHHTTP_STATE	15
3.2.2.4 E_RESULT_PROTO_TYPE_FILE_STATE.....	17
3.3 LCOpenSDK_VideoPlay	18
3.3.1 initVideoPlay	18
3.3.2 uninitVideoPlay.....	18
3.3.3 getIndex.....	18
3.3.4 setIndex	19
3.3.5 setDisplayRegion	19
3.3.6 setPlayerListener	19
3.3.7 getPlayerListener.....	19
3.3.8 playAudio	20

3.3.9	stopAudio	20
3.3.10	seek.....	20
3.3.11	pause.....	20
3.3.12	resume	21
3.3.13	rewake	21
3.3.14	snapShot.....	21
3.3.15	startRecord	21
3.3.16	stopRecord	22
3.3.17	setStreamCallback	22
3.3.18	doScale	22
3.3.19	getScale	23
3.3.20	translate.....	23
3.3.21	getTranslateX	23
3.3.22	getTranslateY	24
3.3.23	setMaxScale	24
3.3.24	setIdentity	24
3.3.25	getCurrentStreamMode.....	24
3.3.26	playRtspReal.....	25
3.3.27	playDeviceRecord.....	25
3.3.28	playCloud.....	26
3.3.29	stopDeviceRecord	27
3.3.30	stopCloud	27
3.3.31	stopRtspReal	28
3.3.32	setSEnhanceMode	28
3.3.33	isRecording.....	29
3.3.34	isStreamPlayed	29
3.3.35	getPlayerStatus.....	29
3.3.36	setKey	30
3.3.37	setDecodeEngine	30
3.4	LCOpenSDK_AudioTalk.....	31

3.4.1	setTalkerListener.....	31
3.4.2	getTalkerListener	31
3.4.3	playTalk	31
3.4.4	stopTalk	32
3.4.5	playSound.....	32
3.4.6	stopSound	32
3.4.7	startSampleAudio	33
3.4.8	stopSampleAudio.....	33
3.5	LCOpenSDK_Utils.....	33
3.5.1	getSaveStreamPath.....	33
3.5.2	computeMD5Lower	34
3.5.3	computeMD5Upper	34
3.5.4	hmac_sha512.....	34
3.5.5	sha512Cry	35
3.5.6	compute1.....	35
3.5.7	compute2	36
3.5.8	getMac.....	36
3.5.9	getOsVersion.....	36
3.5.10	loadSubModules.....	37
3.5.11	setMobileLogLevel.....	37
3.5.12	setSaveStreamFlag	37
3.6	LCOpenSDK_EventListener (Play Service and Window Event Callback)	38
3.6.1	onPlayerResult	38
3.6.2	onResolutionChanged.....	38
3.6.3	onReceiveData	38
3.6.4	on PlayBegan	39
3.6.5	onPlayFinished	39
3.6.6	onPlayerTime	39
3.6.7	onBadFile.....	40
3.6.8	onFrameLost	40

3.6.9	onRecordStop	40
3.6.10	onFileTime	41
3.6.11	onPlayerTimeAndStamp	41
3.6.12	onAudioTimeAndStamp	41
3.6.13	onStreamLogInfo	42
3.7	LCOpenSDK_TalkerListener	42
3.7.1	onTalkPlayReady	42
3.7.2	onTalkResult	42
3.8	LCOpenSDK_Download	43
3.8.1	startDownload	43
3.8.2	startCloudDownload	43
3.8.3	stopDownload	44
3.9	LCOpenSDK_DownloadListener	45
3.9.1	onDownloadReceiveData	45
3.9.2	onDownloadState	45
3.9.3	onDownloadProgress	45
4	SDK Connection Description	46
4.1	SDK Integration Process and Key Business Interface Calling	46
4.1.1	SDK Connection Process	46
4.1.2	Key Business Interface Calling	47
4.1.2.1	Live View	47
4.1.2.2	HD/SD Switching	48
4.1.2.3	Local Recording	49
4.1.2.4	Capture	49
4.1.2.5	PTZ Control	49
4.1.2.6	Voice Intercom	50
4.1.2.7	Playing Back Device Recordings	51
4.1.2.8	Download Device Recordings	52
4.1.2.9	ARC talk	53
4.1.2.10	Enable P2P	53
4.1.2.11	Low-power devices	54
4.2	Demo Features	55
4.2.1	Background	55
4.2.2	Function Description and Instructions	55

4.2.2.1	Account Login	55
4.2.2.2	Obtaining Device List	56
4.2.2.3	Live View and Voice Intercom	56
4.2.2.4	PTZ Control	57
4.2.2.5	Local Video Playback.....	57
4.2.2.6	Device Record Download	58
4.2.2.7	Cloud Record Playback.....	59
4.2.2.8	Cloud Record Download	60



1 Overview

This document describes how to integrate LCOpenSDK from the following aspects and takes Windows as an example.

<u>Configuration of Development Environment</u>	This part introduces how to add PC SDK to projects in Visual Studio and demonstrate by building Demo.
<u>SDK Interface Description</u>	This part describes the functions, parameters, return values, and precautions of all interfaces of LCOpenSDK.
<u>SDK Connection Description</u>	This part describes the process, key business, interface call of SDK docking, as well as demo building.



2 Configuration of Development Environment

2.1 SDK Product

Product	Interface	Description
LCOpenSDKmd.dll	Provides a series of static libraries.	Video play control services (live videos, playbacks, recording, capturing, dragging, pausing and restoring, scaling, translation, fisheye, and more)
		Voice talk
		Platform interaction
		Livestreaming
		Encryption and decryption
		The related header files of the businesses above
		Load according to different application protocols
OpenSDK /include	Header file contents	
Other dependency libraries	LCOpenApiClient md.dll and more	Protocol library

Library Description

Library name	Whether required or not	Description
LCOpenSDKmd.dll	Required	The overall SDK library implements common basic component capabilities and dynamically loads service libraries.

2.2 Adding SDK Package

You need to add dynamic dll dependencies to the project.

2.3 Visual Studio Project Configuration

- 1) Path: File > Open > Project/Solution
- 2) Add header: Project Properties > Configuration > Configuration properties > C/C++ > general > Additional Package Directory.
Add the header file directory in it.
- 3) Add library: Project Properties > Configuration > Configuration Properties > Linker > General > Additional Library Directory.
Add the library file directory in it.



3 SDK Interface Description

3.1 Overview

The open platform SDK is released in the form of dynamic libraries (including .h files, .dll or .lib files). It provides multiple methods: LCOpenSDK_VideoPlay (video play), LCOpenSDK_AudioTalk (voice intercom), and LCOpenSDK_Utils (public method). Applications only need to integrate the above methods to quickly integrate video capabilities.

In addition, LCOpenSDK_PlayerListener and LCOpenSDK_TalkerListener listening interfaces are provided, which are used with LCOpenSDK_VideoPlay and LCOpenSDK_AudioTalk respectively. Applications can inherit and set corresponding listening interfaces to receive play and service callback during intercom. Service callbacks include play window events, status codes during video play and voice intercom, decoding timestamp, and more. For more information about callback interfaces, see the following instructions.

3.2 LCOpenSDK_Define

3.2.1 Parameter Enumeration

3.2.1.1 E_STREAM_TYPE

Methods	E_STREAM_TYPE	
Overview	Stream type enumeration	
Value	STREAM_TYPE_MAIN	Main stream
	STREAM_TYPE_MINOR_1	Sub stream 1
	STREAM_TYPE_MINOR_2	Sub stream 2
	STREAM_TYPE_MINOR_3	Sub stream 3
Remarks		

3.2.1.2 E_MEDIA_FORMAT

Methods E_MEDIA_FORMAT		
Overview	Video format enumeration	
Value	E_MEDIA_FORMAT_MEDIA_DAV	DAV format
	E_MEDIA_FORMAT_MEDIA_MP4	MP4 format
	E_MEDIA_FORMAT_MEDIA_AVI	AVI format
	E_MEDIA_FORMAT_MEDIA_ASF	ASF format
	E_MEDIA_FORMAT_MEDIA_FLV	FLV format
	E_MEDIA_FORMAT_MEDIA_MOV	MOV format
	E_MEDIA_FORMAT_MEDIA_MP464	MP464 format
	E_MEDIA_FORMAT_MEDIA_MOV64	MOV64 format
	E_MEDIA_FORMAT_MEDIA_MP4NOSEEK	MP4NOSEEK format
	E_MEDIA_FORMAT_MEDIA_WAV	WAV format
Remarks		

3.2.1.3 E_MEDIA_TYPE

Methods E_MEDIA_TYPE		
Overview	Media type	
Value	E_MEDIA_TYPE_VIDEO	Video
	E_MEDIA_TYPE_AUDIO	Audio
	E_MEDIA_TYPE_ALL	Video + Audio
Remarks		

3.2.1.4 E_LOG_LEVEL

Methods E_LOG_LEVEL	
Overview	Log level

Value	E_LOG_LEVEL_FAULT	Fatal errors.
	E_LOG_LEVEL_ERR	Errors
	E_LOG_LEVEL_WARNING	Might result in errors.
	E_LOG_LEVEL_INFO	Current running status
	E_LOG_LEVEL_DEBUG	Detailed debugging information
	E_LOG_LEVEL_ALL	All log information
Remarks		

3.2.1.5 E_DEVICE_STATE

Methods E_DEVICE_STATE		
Overview	Device status	
Value	E_DEVICE_STATE_ON_LINE	Online
	E_DEVICE_STATE_OFF_LINE	Offline
	E_DEVICE_STATE_UNKNOWN	Unknown
Remarks		

3. 2. 2 Callback Enumeration

3.2.2.1 E_RESULT_PROTO_TYPE

Methods E_RESULT_PROTO_TYPE		
Overview	onPlayerResult callback type parameter enumeration (service type)	
Value	E_RESULT_PROTO_TYPE_RTSP	RTSP service
	E_RESULT_PROTO_TYPE_HLS	HLS service
	E_RESULT_PROTO_TYPE_FILE	FILE service
	E_RESULT_PROTO_TYPE_NETSDK	NETSDK service
	E_RESULT_PROTO_TYPE_SIP	SIP service
	E_RESULT_PROTO_TYPE_DHHTTP	Optimized streams
	E_RESULT_PROTO_TYPE_DPSDK	acquiring for HTTP

	E_RESULT_PROTO_TYPE_Express E_RESULT_PROTO_TYPE_CloudBase E_RESULT_PROTO_TYPE_RTP E_RESULT_PROTO_TYPE_FINANCECLOUD E_RESULT_PROTO_TYPE_COMMUNITYCLOUD E_RESULT_PROTO_TYPE_MSP E_RESULT_PROTO_TYPE_P2P E_RESULT_PROTO_TYPE_REST	DPSDK service Express service Video cloud service RTP service CMB financial cloud service Community cloud service P2P service MSP service P2P service REST service
Remarks	<i>RTSP service (includes live view, Local videos playback and voice talk)</i> <i>HLS service (includes cloud videos playback and download)</i> <i>FILE service (includes local files playback)</i> <i>NETSDK service (includes P2P device live view and videos playback)</i> <i>SIP service</i> <i>REST service (involves RTSP, HLS, and SIP services)</i> <i>DPSDK, Express, CloudBase are industry platform service</i>	

3.2.2.2 E_RESULT_PROTO_TYPE_RTSP_STATE

Methods	E_RESULT_PROTO_TYPE_RTSP	
Overview	Code related to E_RESULT_PROTO_TYPE_RTSP_STATE service	
Value	E_RESULT_PROTO_TYPE_RTSP_STATE_PACKET_COMPONENT_ERROR E_RESULT_PROTO_TYPE_RTSP_STATE_FRAME_ERROR E_RESULT_PROTO_TYPE_RTSP_STATE_FRAMING_FAILED	Failed to call the streaming media interface in the component Framing failed.

TE_RTSP_TEARDOWN_ERROR	
E_RESULT_PROTO_TYPE_RTSP_STA	Disabling is required
TE_RTSP_DESCRIBE_READY	internally, such as
E_RESULT_PROTO_TYPE_RTSP_STA	disconnection.
TE_RTSP_AUTHORIZATION_FAIL	
E_RESULT_PROTO_TYPE_RTSP_STA	The session has received
TE_RTSP_PLAY_READY	Describe response.
E_RESULT_PROTO_TYPE_RTSP_STA	
TE_RTSP_FILE_PLAY_OVER	RTSP authentication failed.
E_RESULT_PROTO_TYPE_RTSP_STA	
TE_RTSP_PAUSE_READY	Received play response.
E_RESULT_PROTO_TYPE_RTSP_STA	
TE_RTSP_KEY_MISMATCH	Recording file playback ends
E_RESULT_PROTO_TYPE_RTSP_STA	normally.
TE_RTSP_LIVE_PAUSE_ENABLE	
E_RESULT_PROTO_TYPE_RTSP_STA	Received PAUSE response
TE_RTSP_LIVE_PAUSE_DISABLE	
E_RESULT_PROTO_TYPE_RTSP_STA	Secret key error.
TE_RTSP_TALK_BUSY_LINE	
E_RESULT_PROTO_TYPE_RTSP_STA	Supports PAUSE on the server
TE_RTSP_TALK_CHECK_FAILED	livestream.
E_RESULT_PROTO_TYPE_RTSP_STA	
TE_RTSP_SERVICE_UNAVAILABLE	Not support PAUSE on the
E_RESULT_PROTO_TYPE_RTSP_STA	server livestream.
TE_RTSP_USER_INFO_BASE_START	
E_RESULT_PROTO_TYPE_RTSP_STA	Voice talk is busy.
TE_RTSP_STREAM_LIMIT_NOTIFY	
E_RESULT_PROTO_TYPE_RTSP_STA	Operation conditions for voice
TE_RTSP_CONCURRENT_LIMIT_NOT	talk not reached.

	IFY	Maximum number of connection errors (based on 503). Start code for user information. Notification for data usage limit Concurrent limit notification.
Remarks		

3.2.2.3 E_RESULT_PROTO_TYPE_DHHTTP_STATE

Methods E_RESULT_PROTO_TYPE_DHHTTP		
Overview	Code related to E_RESULT_PROTO_TYPE_DHHTTP_STATE service	
Value	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHHTTP_COMPONENT_ERROR	"-1", failed to call the streaming media interface in the component.
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHHTTP_OK	"1000", Enable Play successfully.
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHHTTP_PLAY_FILE_OVER	"2000", the current file finished playing after playback.
	E_RESULT_PROTO_TYPE_DHHTTP_S	

TATE_DHHTTP_BAD_REQUEST	"400000" illegal request, close the client.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_UNAUTHORIZED	"401000" unauthorized,
E_RESULT_PROTO_TYPE_DHHTTP_S	incorrect username and password.
TATE_DHHTTP_FORBIDDEN	
E_RESULT_PROTO_TYPE_DHHTTP_S	"403000" forbidden access,
TATE_DHHTTP_NOTFOUND	close the client.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_REQ_TIMEOUT	"404000" not found, close the client.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_SERVER_ERROR	"408000" Request timeout, indicating successful streaming, but subsequent network errors lead to disconnection of the stream.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_SERVER_UNVALILABLE	"500000" Internal server error, close the client.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_FLOWLIMIT	"503000" Service unavailable.
E_RESULT_PROTO_TYPE_DHHTTP_S	
TATE_DHHTTP_P2P_MAXCONNECT	"503006" mts data limit.
E_RESULT_PROTO_TYPE_DHHTTP_S	

	TATE_DHHTTP_GATEWAY_TIMEOUT E_RESULT_PROTO_TYPE_DHHTTP_S TATE_DHHTTP_CLIENT_ERROR E_RESULT_PROTO_TYPE_DHHTTP_S TATE_DHHTTP_KEY_ERROR	"503007" p2p has reached the maximum number of connections. "504000" Network not connected. "1000000" Internal client error, general code logic error. "1000005" The client secret key mismatch the server key.
Remarks		

3.2.2.4 E_RESULT_PROTO_TYPE_FILE_STATE

Methods	E_RESULT_PROTO_TYPE_FILE	
Overview	Code related to E_RESULT_PROTO_TYPE_FILE service.	
Value	E_RESULT_PROTO_TYPE_FILE_INIT_FAILE D E_RESULT_PROTO_TYPE_FILE_SEEK_FAIL ED E_RESULT_PROTO_TYPE_FILE_SEEK_SUC CESS	"-1" Local file playback initialization failed. "6" Dragging failed. "7" Dragging succeeded. "8" Dragging out of bounds.

	E_RESULT_PROTO_TYPE_FILE_CROSS_B ORDER	
Remarks		

3.3 LCOpenSDK_VideoPlay

3.3.1 initVideoPlay

Methods	int initVideoPlay(int index, void* hWnd);
Overview	Initialize the play window.
Parameters	Index Unique ID of the play window, which is passed in from the outer layer. hWnd Specify the handle of the play window.
Return value	int
Remarks	None

3.3.2 uninitVideoPlay

Methods	void uninitVideoPlay();
Overview	initVideoPlay reversed interface to release internal resources.
Parameter	None
Return value	None
Remarks	None

3.3.3 getIndex

Methods	int getIndex();
Overview	Get current window index.
Parameter	None
Return value	Current window index value
Remarks	None

3.3.4 setIndex

Methods	void setIndex(int index);
Overview	Set the current window index.
Parameter	Index value
Return	None
Value	
Remarks	None

3.3.5 setDisplayRegion

Methods	bool setDisplayRegion(DISPLAY_RECT *srcRect, bool enable);
Overview	Set or increase the display area.
Parameter	srcRect Partial display area enable Enable (settings) or disable the display area
Return	If succeeded, return True. If failed, return False.
Value	
Remarks	Can be enlarged partially.

3.3.6 setPlayerListener

Methods	void setPlayerListener(LCOpenSDK_PlayerListener* playerListener);
Overview	Set the play and window events listening.
Parameter	playerListener Listening pointer
Return	None
Value	
Remarks	None

3.3.7 getPlayerListener

Methods	(LCOpenSDK_PlayerListener*) getPlayerListener();
Overview	Get play window listener.
Parameter	None
Return value	Listening pointer
Remarks	None

3.3.8 playAudio

Methods	int playAudio();
Overview	Play audio.
Parameter	None
Return value	-1 indicates failure. 0 indicates success.
Remarks	None

3.3.9 stopAudio

Methods	int stopAudio();
Overview	Disable audio.
Parameter	None
Return value	-1 indicates failure. 0 indicates success.
Remarks	None

3.3.10 seek

Methods	int seek(long seconds);
Overview	Video playback positioning function.
Parameter	OffsetTime: Offset time relative to the start time of recording.
Return Value	0 indicates callback was successful, and non-zero numbers indicate callback failed. Whether the operation is successful or not needs to be determined based on the status code of the onPlayerResult callback. Please refer to the service code definition for reference.
Remarks	Asynchronous interface.

3.3.11 pause

Methods	-(int) pause;
Overview	Pause play.
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed. Whether the operation is successful or not needs to be determined based on the status code of the onPlayerResult callback. Please refer to the service code definition for reference.
Remarks	Pause play.

3.3.12 resume

Methods	int resume();
Overview	Resume video playback (pause reverse interface).
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed. Whether the operation is successful or not needs to be determined based on the status code of the onPlayerResult callback. Please refer to the service code definition for reference.
Remarks	To restore playback.

3.3.13 awake

Methods	int awake();
Overview	Interrupt dormancy
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	To restore playback.

3.3.14 snapShot

Methods	int snapShot(const char* filePath);
Overview	Local capture.
Parameter	filePath: Image file name (including path)
Return Value	-1 indicates failure. 0 indicates success.
Remarks	You can call this interface to save a frame of image as a local JPG image during playback.

3.3.15 startRecord

Methods	int startRecord(const char* filePath, int recordType, long long spaceRemain, int convertMode);
Overview	Start recording locally.
Parameter	File path: The file path nRecordType: Recording format (refer to LCOpenSDK_Define E_MEDIA_FORMAT definition) spaceRemain: The remaining disk capacity (default is 0xFFFFFFFF).

	convertMode: The save mode for recorded files (default is 1). 0 represents recording as a single file. 1 represents recording multiple files when parameters such as stream resolution, encoding format, frame rate, etc., change.
	0 indicates call was successful, and non-zero numbers indicate call failed.
Remarks	You can call this interface to record a video to the local file during playback.

3.3.16 stopRecord

Methods	int stopRecord();
Overview	Stop recording locally.
Parameter	None
Return value	-1 indicates failure. 0 indicates success.
Remarks	Use together with startRecord.

3.3.17 setStreamCallback

Methods	void setStreamCallback(OUTPUT_STREAM_FORMAT streamFormat);
Overview	Set the stream callback.
Parameter	streamFormat: The output streaming formats currently supported are DAV, PS, and TS. PS: 1, TS: 2, DAV.
Return Value	None
Remarks	1. You can call this interface to enable the stream callback before or during playback. 2. The streams are exported through LCOpenSDK_PlayListener::onStreamCallback interface. Each time a frame of video or audio is exported, the first frame is a video I-frame. 3. You can set the output streaming format to OUTPUT_STREAM_FORMAT_NULL to disable it when the stream callback is not needed.

3.3.18 doScale

Methods	void doScale(float scale);
Overview	Video electronic zoom.

Parameter	scale	Relative scaling ratio.
Return value	None	
Remarks		

3.3.19 getScale

Methods	float getScale();	
Overview	Get scaling ratio.	
Parameter	None	
Return value	Scaling ratio of normal video images.	
Remarks		

3.3.20 translate

Methods	void translate(float x, float y);	
Overview	Window movement.	
Parameter	x	The absolute horizontal coordinate of the destination position point with respect to the top left corner of the screen.
	y	The absolute vertical coordinate of the destination position point with respect to the top left corner of the screen.
Return Value	None	
Remarks	None	

3.3.21 getTranslateX

Methods	float getTranslateX();	
Overview	Get the window moving distance (x coordinate).	
Parameter	None	
Return value	Moving distance	
Memo		

3.3.22 getTranslateY

Methods	float getTranslateY();
Overview	Get the window moving distance (y coordinate).
Parameter	None
Return value	Moving distance
Memo	

3.3.23 setMaxScale

Methods	void setMaxScale(float maxScale);
Overview	Set the maximum scaling ratio of the screen.
Parameter	maxScale Maximum scaling ratio
Return Value	None
Remarks	

3.3.24 setIdentity

Methods	void setIdentity();
Overview	Reset screen zoom and translation operations.
Parameter	None
Return value	None
Remarks	

3.3.25 getCurrentStreamMode

Methods	E_STREAM_MODE getCurrentStreamMode();
Overview	Get current stream acquiring type.
Parameter	None

Return value	Current stream type (see E_STREAM_MODE definition)
Remarks	We recommend calling this interface after receiving the OnPlayerBegan callback.

3. 3. 26 playRtspReal

Methods	int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int channelId, int definitionMode, bool isOpt, const char* playInfo, bool isTryP2P);
Overview	Play live stream.
Parameter	token: User play token; deviceId: Device serial number; decryptKey: Stream decryption key; channelId: Channel number; definitionMode: Stream type, 0 indicates main stream and 1 indicates sub stream; isOpt: Whether to enable long links; playInfo: Playback information; isTryP2P: Whether to enable p2p.
Return Value	0 / Non-0 Success/failure. The onPlayerResult callback service code determines whether the play succeeds.
Remarks	None

3. 3. 27 playDeviceRecord

Methods	int playDeviceRecord(const char* token, const char* deviceId, const char* decryptKey, int channelId, const char* fileId, int beginTime, int endTime, int streamType, const char* playInfo, bool isTryP2P);
Overview	Play local record.

Parameter	token: User play token; deviceID: Device serial number; decryptKey: Stream decryption key; channelID: Channel number; fileID: The file name of the video, which is used for playback by file; beginTime: The start time of the video, which is used for playback by time; endTime: The end time of the video, which is used for playback by time; streamType: Stream type, 0 indicates main stream and 1 indicates sub stream; playInfo: Playback information; isTryP2P: Whether to enable p2p.
Return Value	0 / Non-0 Success/failure. The onPlayerResult callback service code determines whether the play succeeds.
Remarks	None

3.3.28 playCloud

Methods	int playCloud(const char* deviceID, int channelID, const char* decryptKey, const char* recordId, int offsetTime, const char* playInfo, const char* signatureUrl, const char* recordPath, const char* recordRegionId);
Overview	Play cloud stream.
Parameter	deviceID: Device serial number; channelID: Channel number; decryptKey: Stream decryption key;

	recordId: Record file id; offsetTime: offset Time; playInfo: Playback information; signatureUrl: The download link for the M3U file returned by the cloud video recording query; recordPath: Location for storing downloaded files; recordRegionId: The region ID returned by the cloud video recording query.
Return	o / Non-o Success/failure.
Value	The onPlayerResult callback service code determines whether the play succeeds.
Remarks	None

3.3.29 stopDeviceRecord

Methods	int stopDeviceRecord();
Overview	Stop play device record.
Parameter	None
Return value	o indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	Call this interface when all the play stops.

3.3.30 stopCloud

Methods	int stopCloud();
Overview	Stop play cloud record.
Parameter	None
Return	o indicates callback was successful, and non-zero numbers indicate

value	callback failed.
Remarks	Call this interface when all the play stops.

3.3.31 stopRtspReal

Methods	int stopRtspReal();
Overview	Stop playing.
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	Call this interface when all the play stops.

3.3.32 setSEnhanceMode

Methods	bool setSEnhanceMode(intmode);
Overview	Set the noise reduction mode level (takes effect after playAudio is called).
Parameter	None
Return value	True: Success; False: Failure.
Remarks	mode(-1,0,1,2,3,4) -1 indicates no noise reduction. 0 indicates the lowest noise reduction level, and the minimum damage to available voice signals. 4 indicates the maximum degree of noise reduction level, and the maximum damage to available voice signals.

3.3.33 isRecording

Methods	bool isRecording();
Overview	Whether in recording mode
Parameter	None
Return value	Whether recording now
Remarks	

3.3.34 isStreamPlayed

Methods	bool isStreamPlayed();
Overview	Whether in playing mode
Parameter	None
Return value	Whether playing now
Remarks	

3.3.35 getPlayerStatus

Methods	int getPlayerStatus();
Overview	Get play status.
Parameter	None
Return value	STATUS_PLAYING 0 STATUS_STOPED 1 STATUS_PAUSE 2 STATUS_REQUESTING 3 STATUS_FAILED 4 STATUS_SEEKING 5 STATUS_ONE_RECORD_FINISHED 6
Remarks	

3. 3. 36 setKey

Methods	void setKey(const char* passwordKey);
Overview	Update audio and video decryption key.
Parameter	Passwordkey Product secret key
Return value	None
Remarks	

3. 3. 37 setDecodeEngine

Methods	void setDecodeEngine(int engineType);												
Overview	Set the video decoding engine.												
Parameter	<table><tr><td>DECODE_NOTSET = 0</td><td>Not set</td></tr><tr><td>DECODE_SW</td><td>Software decoding</td></tr><tr><td>DECODE_HW</td><td>Hardware decoding copying mode (use d3d9 interface when using windows platform).</td></tr><tr><td>DECODE_HW_FAST</td><td>Hardware decoding displaying mode (use d3d9 interface when using windows platform).</td></tr><tr><td>DECODE_MSDK</td><td>Hardware decoding by calling intel media sdk, discarded.</td></tr><tr><td>DECODE_HW_FAST_D3D11</td><td>Hardware decoding displaying mode (use d3d9 interface only for windows platform).</td></tr></table>	DECODE_NOTSET = 0	Not set	DECODE_SW	Software decoding	DECODE_HW	Hardware decoding copying mode (use d3d9 interface when using windows platform).	DECODE_HW_FAST	Hardware decoding displaying mode (use d3d9 interface when using windows platform).	DECODE_MSDK	Hardware decoding by calling intel media sdk, discarded.	DECODE_HW_FAST_D3D11	Hardware decoding displaying mode (use d3d9 interface only for windows platform).
DECODE_NOTSET = 0	Not set												
DECODE_SW	Software decoding												
DECODE_HW	Hardware decoding copying mode (use d3d9 interface when using windows platform).												
DECODE_HW_FAST	Hardware decoding displaying mode (use d3d9 interface when using windows platform).												
DECODE_MSDK	Hardware decoding by calling intel media sdk, discarded.												
DECODE_HW_FAST_D3D11	Hardware decoding displaying mode (use d3d9 interface only for windows platform).												
Return value	None												
Remarks													

3.4 LCOpenSDK_AudioTalk

3.4.1 setTalkerListener

Methods	void setTalkerListener(LCOpenSDK_TalkerListener* talkerListener);	
Overview	Set listening for voice talk.	
Parameter	talkerListener	Listening pointer
Return Value	None	
Remarks	The listener needs to implement LCOpenSDK_TalkerListener and can receive the status code callback of the voice intercom.	

3.4.2 getTalkerListener

Methods	LCOpenSDK_TalkerListener* getTalkerListener;	
Overview	Get the listener for voice intercom listening.	
Parameter	None	
Return value	Listening pointer	
Remarks	None	

3.4.3 playTalk

Methods	int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo, bool isTryP2P);	
Overview	Start voice talk.	
Parameter	token: User play token; deviceId: Device serial number; channelID: Channel number (transmit -1 for device intercom, and the corresponding channel number for channel intercom). isOpt: Whether to enable long links. decryptKey: Stream decryption key; playInfo: Playback information;	

	isTryP2P: Whether to enable p2p.
Return Value	0 indicates callback was successful, and non-zero numbers indicate callback failed. Whether the actual intercom is successful depends on callback status code, and you need to callback to the listener interface.
Remarks	1. Set listening before enabling voice talk to receive callback of the service code.

3. 4. 4 stopTalk

Methods	int stopTalk();
Overview	Stop voice talk.
Parameter	None
Return value	The interface always returns 0.
Remarks	None

3. 4. 5 playSound

Methods	int playSound();
Overview	Enable voice talk play.
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	Enable voice talk audio.

3. 4. 6 stopSound

Methods	int stopSound();
Overview	Disable voice talk.
Parameter	None
Return	0 indicates callback was successful, and non-zero numbers indicate

value	callback failed.
Remarks	Disable voice talk audio.

3.4.7 startSampleAudio

Methods	int startSampleAudio();
Overview	Start audio collection.
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	This interface needs to be called after the successful callback of the voice talk in order to be effective.

3.4.8 stopSampleAudio

Methods	NSInteger stopSampleAudio();
Overview	Stop audio collection.
Parameter	None
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	This interface needs to be called before stopping the talk.

3.5 LCOpenSDK_Utils

3.5.1 getSaveStreamPath

Methods	const char* getSaveStreamPath();
Overview	Get stream file path.
Parameter	None
Return	File path

value	
Remarks	None

3.5.2 computeMD5Lower

Methods	Void computeMD5Lower(const char* srcStr, char outStr[128]);
Overview	Compute the MD5 encryption value, and the result needs to be lowercase.
Parameter	srcStr Encrypted value. srcStr Output value after encryption.
Return	None
Value	
Remarks	None

3.5.3 computeMD5Upper

Methods	void computeMD5Upper (const char* srcStr, char outStr[128]);
Overview	Compute the MD5 encryption value, and the result needs to be uppercase.
Parameter	srcStr Encrypted value. srcStr Value after encryption.
Return	None
Value	
Remarks	None

3.5.4 hmac_sha512

Methods	char * hmac_sha512(const char * key, const char * data, char outData[128]);
Overview	HMAC-SHA512 encryption.

Parameter	key	Encryption key.
	data	Encrypted data.
	outData	Encrypted value
Return	None	
Value		
Remarks	None	

3.5.5 sha512Cry

Methods	char * sha512Cry(const char * data);	
Overview	SHA512 Encryption.	
Parameter	data	Encrypted data.
Return	Encrypted data	
Value		
Remarks	The new character array is displayed inside the interface. Remember to release the returned value after it is used up.	

3.5.6 compute1

Methods	void compute1(const char* srcStr, char outStr[128]);	
Overview	Base64 MD5 lowercase encryption.	
Parameter	srcStr	Data to be encrypted
	outStr	Encrypted data
Return	None	
Value		
Remarks	None	

3.5.7 compute2

Methods	void compute2(const char* srcStr, char outStr[128]);	
Overview	Base64 MD5 uppercase encryption.	
Parameter	srcStr	Data to be encrypted
	outStr	Encrypted data
Return	None	
Value		
Remarks	None	

3.5.8 getMac

Methods	void getMac(char outMac[128]);	
Overview	Get device MAC address.	
Parameter	outMac	Input, which can be an empty character array.
Return	outMac	The output MAC address.
Value		
Remarks	None	

3.5.9 getOsVersion

Methods	void getOsVersion(char version[128]);	
Overview	Get OS version.	
Parameter	version	Input, which can be an empty character array.
Return	version	The output OS version information.
Value		
Remarks	None	

3.5.10 loadSubModules

Methods	void loadSubModules();
Overview	Loading media sub-inventory.
Parameter	None
Return value	None
Remarks	Media sub-inventory must be loaded before the business.

3.5.11 setMobileLogLevel

Methods	void SetMobileLogLevel(OpenSDK_LogLevel level = All);
Overview	Set the log output level.
Parameter	logLevel Output log level (refer to OpenSDK_LogLevel)
Return Value	None
Remarks	Call this interface before all other interfaces.

3.5.12 setSaveStreamFlag

Methods	void setSaveStreamFlag(bool isOpen);
Overview	Set whether to enable the stream to write a file (playback, voice talk).
Parameter	isOpen YES/NO On/Off
Return Value	None
Remarks	Call this function before starting play (voice talk) to take effect. Do not call this function in the release version, only use it for debugging purposes.

3.6 LCOpenSDK_EventListener (Play Service and Window Event Callback)

If the upper layer needs to receive callbacks for playback services and window events, follow the LCOpenSDK_PlayerListener protocol and set listening through the setPlayerListener interface to receive related callbacks.

3.6.1 onPlayerResult

Methods	<code>void onPlayerResult(int index, const char* code, int resultSource);</code>
Overview	The callback of the video play status.
Parameter	Index: Unique ID of the playback window. Code: Status code Type: Service type
Return Value	None
Remarks	Index: Window index; Code: The code varies depending on the type. See 错误!未找到引用源。 E_RESULT_PROTO_TYPE_DHHTTP_STATE E_RESULT_PROTO_TYPE_FILE_STATE type: See E_RESULT_PROTO_TYPE ; Close the session after receiving the callback error. See demo for details.

3.6.2 onResolutionChanged

Methods	<code>void onResolutionChanged(int index, int width, int height)</code>
Overview	The callback of the resolution change.
Parameter	Index: Unique ID of the play window. Width: Width Height: Height
Return Value	None
Remarks	None

3.6.3 onReceiveData

Methods	<code>void onReceiveData(int index, int len);</code>
---------	--

Overview	Accept callback of the stream data length.
Parameter	Index: Unique ID of the playback window. Len: Data length
Return Value	None
Remarks	You can calculate the code stream rate based on len.

3. 6. 4 on PlayBegan

Methods	void onPlayBegan(int index);	
Overview	Start playing (the first frame has been decoded).	
Parameter	index	Window number.
Return Value	None	
Remarks	When the play starts, this callback event is triggered (including play after pause and play after seek).	

3. 6. 5 onPlayFinished

Methods	void onPlayFinished (int index);	
Overview	Recording play ended.	
Parameter	index	Window number.
Return Value	None	
Remarks	This callback event is triggered when the device recording or local recording playback ends.	

3. 6. 6 onPlayerTime

Methods	void onPlayerTime(int index, long time);	
Overview	Play time callback.	
Parameter	Index: Window serial number	

	Time: UTC time
Return Value	None
Remarks	Call back the current decoding timestamp.

3. 6. 7 onBadFile

Methods	void onBadFile(const char* context);	
Overview	Exception occurred when playing local recording file.	
Parameter	context	context
Return Value	None	
Remarks	Invalid files are detected when local files are played.	

3. 6. 8 onFrameLost

Methods	void onFrameLost(const char* context);	
Overview	Dropping frames callback.	
Parameter	context	context
Return Value	None	
Remarks	None	

3. 6. 9 onRecordStop

Methods	void onRecordStop(const char* context, int error);	
Overview	Recording end callback.	
Parameter	Error Error code (0 indicates normal termination). index Window number. context context	
Return Value	None	

3. 6. 10 onFileTime

Methods	void onFileTime(const char* context, long beginTime, long endTime);
Overview	Start time and end time of a local file.
Parameter	beginTime: Start time. endTime: End time. context: context
Return	None
Value	

3. 6. 11 onPlayerTimeAndStamp

Methods	void onPlayerTimeAndStamp(const char* context, long time, long stampTime);
Overview	Callback for decoding timestamps.
Parameter	Time: UTC time stampTime: Time stamp context Context
Return	None
Value	
Remarks	When the local mp4 file is played, time is the number of seconds that has been played, and the others are UTC time.

3. 6. 12 onAudioTimeAndStamp

Methods	void onAudioTimeAndStamp(const char* context, long time, long stampTime, bool isDetach);
Overview	Callback for audio decoding timestamp.
Parameter	Time: UTC time stampTime: Time stamp context Context isDetach Whether the audio is separated.

Return Value	None
Remarks	

3.6.13 onStreamLogInfo

Methods	void onStreamLogInfo(const char* context, const char* message, unsigned int length);	
Overview	Call back stream acquiring log information.	
Parameter	Context	Context
	Message	Log content
	length	length
Return Value	None	
Remarks	None	

3.7 LCOpenSDK_TalkerListener

3.7.1 onTalkPlayReady

Methods	void onTalkPlayReady();
Overview	Callback for the status code of successful voice talk.
Parameter	None
Return value	None
Remarks	

3.7.2 onTalkResult

Methods	void onTalkResult(const char* error, int type);
Overview	Status code callback for voice talk.
Parameter	Error: Status code Type: Service type
Return Value	None
Remarks	Error: It varies according to different types, see 错误!未找到引用源。 E_RESULT_PROTO_TYPE_RTSP_STATE、 错误!未找到引用源。 E_RESULT_PROTO_TYPE_DHHTTP_STATE; type: See 3.2.2.1 E_RESULT_PROTO_TYPE; Close the session after receiving the callback error. See demo for details.

3.8 LCOpenSDK_Download

3.8.1 startDownload

Methods	<code>int startDownload(int index, const char* token, const char* filepath, const char* deviceId, const char* decryptKey, const char* fileId, int beginTime, int endTime, const char* playInfo, int channelId);</code>
Overview	Download device record
Parameter	index: Unique identifier ID for download; token: User play token; filePath: Location for storing downloaded files; deviceId: Device serial number; decryptKey: Stream decryption key; fileId: Files to be downloaded; beginTime: Download start time; endTime: Download end time; playInfo: Playback information; channelId: Channel number;
Return Value	0 / Non-0 Success/failure. The onPlayerResult callback service code determines whether the download succeeds.
Remarks	None

3.8.2 startCloudDownload

Methods	<code>int startCloudDownload(int index, const char* filepath, const char* deviceId, int channelId, const char* recordId, int needBeginTime, int needEndTime, int offsetTime, const char* playInfo, const char* signatureUrl, const char* recordPath, const char* recordRegionId);</code>
Overview	Download cloud record

Parameter	index: Unique identifier ID for download; filePath: Location for storing downloaded files; deviceId: Device serial number; channelID: Channel number; recordID: Files to be downloaded; needBeginTime: Download start time; needEndTime: Download end time; offsetTime: offset time; playInfo: Playback information; signatureUrl: The download link for the M3U file returned by the cloud video recording query; recordPath: Location for storing downloaded files; recordRegionId: The region ID returned by the cloud video recording query.
Return Value	0 / Non-0 Success/failure. The onPlayerResult callback service code determines whether the download succeeds.
Remarks	None

3. 8. 3 stopDownload

Methods	int stopDownload(int index);
Overview	Stop downloading.
Parameter	Index: Unique identifier ID for download;
Return value	0 indicates callback was successful, and non-zero numbers indicate callback failed.
Remarks	None

3.9 LCOpenSDK_DownloadListener

3.9.1 onDownloadReceiveData

Methods	<code>void onDownlaodReceiveData(int index, int datalen);</code>
Overview	Accept callback of the stream data length.
Parameter	Index: Unique ID of the download Len: Data length
Return Value	None
Remarks	You can calculate the code stream rate based on len.

3.9.2 onDownloadState

Methods	<code>void onDownlaodState(int index, const char* code, int type);</code>
Overview	The callback of the download status.
Parameter	Index: Unique ID of the download Code: Status code Type: Service type
Return Value	None
Remarks	Index: Window index; Code: The code varies depending on the type. See 错误!未找到引用源。 E_RESULT_PROTO_TYPE_DHHTTP_STATE E_RESULT_PROTO_TYPE_FILE_STATE type: See E_RESULT_PROTO_TYPE ; Close the session after receiving the callback error. See demo for details.

3.9.3 onDownloadProgress

Methods	<code>void onDownlaodProgress(int index, float progress, int type);</code>
Overview	The callback of the download status.
Parameter	Index: Unique ID of the playback window. Progress: Download progress Type: Service type
Return Value	None
Remarks	Index: Download index; Code: The code varies depending on the type. See 错误!未找到引用源。 E_RESULT_PROTO_TYPE_DHHTTP_STATE E_RESULT_PROTO_TYPE_FILE_STATE

type: See [E_RESULT_PROTO_TYPE](#);

Close the session after receiving the callback error. See demo for details.

4 SDK Connection Description

The PC OpenSDK Demo has already implemented all the core functions currently provided. If developers understand the implementation logic of the source code in the Demo, it will be easy to port these functions into their own applications.

Refer to the PCOpenDemo directory for the source code of the PC Demo project. The project primarily references the PC SDK and implements core video-centric functions around the SDK. You can leverage the provided demo to develop your own client application.

4.1 SDK Integration Process and Key Business Interface Calling

The PC OpenSDK integrates the following main functions: Live view, high/standard definition switching, local recording, capture, PTZ control, voice intercom and video playback.

4.1.1 SDK Connection Process

If your local computer does not have the Visual Studio 2005 environment required to compile the PC SDK, you need to manually include the .manifest files under the IoTApplication_VideoPlay_PC_C_Plus_Plus_V1.1.2\depend\64bit_lib\other, and VC runtime libraries into your project.

The development environment is described in Chapter 2. To initialize the PC SDK and set up the necessary functions, you need to follow a specific sequence of API calls. Below is the initialization process along with explanations for each step:

(1) void LCOpenSdk_Utils::loadSubModules();

Load the media sub-library for streaming to ensure the smooth operation of the streaming process.

```
LCOpenSdk_Utils::loadSubModules();
```

```
(2) void SetMobileLogFile(const char* fileName = "LCOpenSDK", int maxSize = 10*1024*1024, OpenSdk_LogFileMode fileMode = NewFile);
```

Configure the SDK log interface.

```
LCOpenSdk_Utils::SetLogInfo(LCOpenSdk_Utils::All, "DEMO", 10*1024*1024, LCOpenSdk_Utils::RollFile);
```

```
(3) void initOpenApi(const char* host, int port, const char* caPath);
```

Initialize the protocol library, where the parameters host and port are obtained from the developer platform when registering an account. The host is provided by the platform, the port defaults to 443, and caPath is the path to the SDK certificate.

```
m_OpenApiHandle->initOpenApi(param.host.toLatin1().data(), param.port, NULL);
```

4. 1. 2 Key Business Interface Calling

First, obtain the token required for the device list interface: You need to access the API /open-api/api-base/auth/getAppAccessToken (refer to the Dahua Cloud Developer Platform for the specific request and response parameters of this API). For specific usage, you can refer to the function `int TokenCtrl::getIOTToken(QString host, OUT QString& token, QString AccessKey)` in the demo.

Next, to obtain the device list, refer to the Dahua Cloud Developer Platform for the specific request and response parameters of the API /open-api/api-iot/device/getDeviceList. You can also refer to the function `void TokenCtrl::getIOTPlayInfo()` in the demo for implementation details.

4.1.2.1 Live View

Initialization is required before starting playback. You need to call `int initVideoPlay(int index, void* hWnd)` and set up the listener callback `void setPlayerListener(LCOpenSDK_PlayerListener* playerListener)`. (The parameter explanations for these interfaces have been described in Chapter 3. For specific usage, you can refer to the function `int PlayCtrl::init(QWidget *videoBase)` in the demo.)

```

int PlayCtrl::init(QWidget *videoBase)
{
    int ret = 0;
    do
    {
        if (NULL == videoBase)
        {
            ret = -1;
            break;
        }

        m_videoPlay = new LCOpenSDK_VideoPlay();
        if (NULL == m_videoPlay)
        {
            ret = -1;
            break;
        }

        if (0 != m_videoPlay->initVideoPlay(0, (void*)(videoBase->winId())))
        {
            DLOG("LCOpenSDK_VideoPlay init fail!!!\n");
            ret = -2;
            break;
        }
        else
        {
            m_videoPlay->setPlayerListener(this);
        }
    }
}

```

int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int channelId, int definitionMode, bool isOpt, const char* playInfo);

The explanation of the live view interface parameters has been detailed in Chapter 3. It is important to note that the playInfo parameter is obtained from the developer platform's /open-api/api-iot/device/getDeviceList interface. The retrieved playInfo is encrypted and must be decrypted using the SDK's LCOpenSDK_Crypto::decryptIoTPlayInfo decryption interface before being passed into the playRtspReal function. The success or failure of playback can be determined in the callback void onPlayerResult(int index, const char* code, int resultSource), where specific reasons are provided. (You can refer to the callback function implemented in the PlayCtrl class in the demo, which inherits from LCOpenSDK_PlayerListener.)

```

class PlayCtrl : public QObject, public LCOpenSDK_PlayerListener

```

4.1.2.2 HD/SD Switching

int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int channelId, int definitionMode, bool isOpt, const char* playInfo);

definitionMode parameter: 0 represents high definition (HD), and 1 represents standard definition (SD).


```
ret = m_videoPlay->playRtspReal(token.c_str(), m_iotDeviceList->deviceId.c_str(),
    m_iotDeviceList->decryptKey.c_str(), channID, definitionMode, false, playInfo.c_str());
```

4.1.2.3 Local Recording

```
int startRecord(const char* filePath, int recordType, long long spaceRemain = 0xFFFFFFFF,
int convertMode = 1);
```

This interface is used to record the live view screen. You can refer to the implementation in the demo for how to call it.

```
ret = m_videoPlay->startRecord(getRecordPath().toLocal8Bit().data(), 1);
```

4.1.2.4 Capture

```
int snapShot(const char* filePath);
```

This interface is used to capture a screenshot during live view. You can refer to the demo for implementation. Pass in a file path to save the image.

```
ret = m_videoPlay->snapShot(getSnapShotPath().toLocal8Bit().data());
```

4.1.2.5 PTZ Control

This functions uses the OpenApiClient protocol library, and the corresponding header file is located at depend/include/LCOpenApiClient/LCOpenApi/ControlPTZ.h. For the specific parameter passing, you can refer to the implementation in the demo.

```

int OpenApiCtrl::controlPTZ(const PTZ_CONTROL_PARAMETERS &parameters)
{
    ControlPTZRequest *req;
    ControlPTZResponse *resp;

    int ret = 0;
    char iCh[10]={0};
    char strDuration[10] = {0};
    _snprintf(iCh, sizeof(iCh), "%d", parameters.channelId);
    _snprintf(strDuration, sizeof(strDuration), "%d", parameters.duration);
    req = LCOENAPI_INIT(ControlPTZRequest);
    resp = LCOENAPI_INIT(ControlPTZResponse);
    req->data.deviceId = CS(parameters.deviceId.toString().c_str());
    req->data.duration = CS(strDuration);
    req->data.channelId = CS(iCh);
    req->data.h = parameters.horizon;
    req->data.v = parameters.vertical;
    req->data.z = parameters.zoom;
    req->data.operation = CS(parameters.operate.toString().c_str());

    req->data.token = CS(userToken.c_str());

    ret = m_OpenApiHandle->request(req, resp, 10, 0, parameters.playInfo.c_str());
}

```

4.1.2.6 Voice Intercom

2 concepts:

(1) Accompanying audio: The sound that comes along with the video stream, which can be regarded as the audio originating from the device. To enable or disable the audio coming from the device, use the playAudio() and stopAudio() interfaces. (If the device has not enabled the accompanying audio, try enabling the Audio Encode Control through the developer platform interface /open-api/api-iot/device/setAbilityStatus.)

(2) Intercom audio: It is enabled through the playTalk interface. Two-way intercom can be directly achieved through this interface.

There are 2 methods to enable two-way intercom.

Treat the accompanying audio as the intercom audio. After the video rendering is successful, call playAudio() to enable the audio coming from the device. Then, initiate the intercom by calling the int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo) interface. Once the onTalkResult callback confirms the intercom is successful (avoid performing business logic within the callback; instead, start a separate thread), begin audio capture with startSampleAudio();.

```

void RealPlayCtrl1::onTalkResult(const char* error, int type)
{
    DLOG("onTalkResult-%s-%d\n", error, type);
    if (!isNumber(error))
    {
        DLOG("business error[%s]!!!\n", error);
        emit sgnTalkerStatusChanged(STATUS_FAIL);
        return;
    }

    int nError = atoi(error);
    if(0 == type){
        if (STATE_RTSP_DESCRIBE_READY == nError)
        {
            DLOG("talk ready\n");
            return;
        }
        else if (STATE_RTSP_PLAY_READY == nError)
        {
            // start sample audio
            m_thread = std::thread(&RealPlayCtrl1::audioSampletreadFunc, this);
            m_thread.detach();
            DLOG("start to talk\n");
            m_isTalking = true;
        }
    }
}

```

int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo);

Do not treat the accompanying audio as the intercom audio. After the video rendering is successful, do not call playAudio(), which means not enabling the accompanying audio. Only call the playTalk interface to enable intercom, and once the onTalkResult callback indicates the intercom is successful (audio capture needs to be initiated in another thread), proceed with audio capture by calling startSampleAudio(). Call the playSound() interface to enable the audio coming from the device. Remember to call stopSound() when closing the intercom.

4.1.2.7 Playing Back Device Recordings

int playDeviceRecord(const char* token, const char* deviceId, const char* decryptKey, int channelId, const char* fileId, int beginTime, int endTime, int offsetTime, bool isOpt, const char* playInfo);

This interface can be used for both time-based playback and file-based playback.

To play by file, provide the retrieved file ID; you can set beginTime and endTime to 0 or any other value. To play by time, set the fileId to an empty string"" and specify the desired beginTime and endTime for the video segment you want to play.

```

ret = m_videoPlay->playDeviceRecord(token.c_str(), m_recordInfo->deviceId.toStdString().c_str(), m_iotDeviceList->decryptKey.c_str(),
    m_recordInfo->channelId, m_recordInfo->name.toStdString().c_str(), beginTime.toTime_t(), endTime.toTime_t(), 0, false, playInfo.c_str());

```

The interface for querying video information is located in the same folder as the PTZ

interface, specifically in the file QueryLocalRecords.h. For details, refer to the demo code.

```
int RecordListCtrl::getRecordInfo(const QString &deviceId, int channelId, std::string playInfo, const QDate &date)
{
    clearRecordList();

    // QString beginTime = "";
    // QString endTime = "";
    // getBeginTimeAndEndTime(date, beginTime, endTime);
    QString beginTime = date.toString("yyyy-MM-dd").append(" 00:00:00");
    QString endTime = date.toString("yyyy-MM-dd").append(" 23:59:59");

    RECORD_PARAMETERS parameters;
    parameters.deviceId = deviceId;
    parameters.channelId = channelId;
    parameters.beginTime = beginTime;
    parameters.endTime = endTime;
    parameters.playInfo = playInfo;

    int totalRecNum = 0;
    if (DEVICE_RECORD == m_recordType)
    {
        totalRecNum = OpenApiCtrl::getInstance()->getDeviceRecordNum(parameters);
    }
}
```

```
int OpenApiCtrl::getDeviceRecordNum(const RECORD_PARAMETERS &parameters)
{
    if (!m_IsGetTokenOK)
    {
        DLOG("get Token Failed!!!\n");
        return RET_FAIL;
    }

    QueryLocalRecordNumRequest *req;
    QueryLocalRecordNumResponse *resp;
    int ret = 0;
    char strCh[10] = {0};
    _snprintf(strCh, sizeof(strCh), "%d", parameters.channelId);
    req = LCOPENAPI_INIT(QueryLocalRecordNumRequest);
    resp = LCOPENAPI_INIT(QueryLocalRecordNumResponse);

    std::string userToken = "";
    std::vector<DEVICE_INFO> deviceVec = m_tokenCtrl->getDeviceVec();
    for (int i = 0; i < deviceVec.size(); i++)
    {
        // Json::Reader reader;
        // Json::Value root;
        // QJsonParseError jsonError;
        // QJsonDocument json = QJsonDocument::fromJson(deviceVec.at(i).playInfo.c_str(), &jsonError);
        if (jsonError.error != QJsonParseError::NoError)
        {
            QString strError = jsonError.errorString();
            continue;
        }

        if (!json.isObject())
        {
            continue;
        }

        userToken = json.object().value("userToken").toString().toStdString();
        //if (reader.parse(m_tokenCtrl.deviceVec.at(i).playInfo, root)) {
        // userToken = root["userToken"].asString();
        //}
    }
}
```

4.1.2.8 Download Device Recordings

int startDownload(int index, const char* token, const char* filepath, const char* deviceId, const char* decryptKey, const char* fileId, int beginTime, int endTime, const char* playInfo, int channelId);

This interface can be used for downloading either by time or by file.

For downloading by file, the queried 'fileID', 'beginTime', and 'endTime' need to be passed (where 'beginTime' and 'endTime' are internally used to determine the total duration of the downloaded file).

For downloading by time, the 'fileID' should be passed as an empty string '', while 'beginTime' and 'endTime' should specify the desired playback segment of the recording (note that 'beginTime' and 'endTime' must fall within the duration of a single file, as these timestamps are also internally used to determine the total duration of the downloaded file).

The following example demonstrates downloading by file. For specific implementation, please refer to the demo.

```
QDateTime beginTime = QDateTime::fromString(record_info->beginTime, "yyyy-MM-dd HH:mm:ss");
QDateTime endTime = QDateTime::fromString(record_info->endTime, "yyyy-MM-dd HH:mm:ss");
// playInfo
std::string playInfo = iotDeviceList->playInfo;
m_downloadingFilePath = QString(m_deviceRecordsDir + "/" + record_info->deviceId + "_download_" + strBeginTime + ".mp4").toLatin1();
m_lcopensdkDownloadInstance->startDownload(index, record_info->token.toStdString().c_str(), m_downloadingFilePath.data(), iotDeviceList->deviceId.c_str(),
iotDeviceList->decryptKey.c_str(), record_info->name.toStdString().c_str(), beginTime.toTime_t(), endTime.toTime_t(), playInfo.c_str(), 0);
```

4.1.2.9 ARC talk

Currently, OpenSDK supports two-way intercom functionality. Audio capture permissions need to be requested before enabling intercom. The ARC intercom integration process is as follows: Call the developer platform interface (/open-api/api-converter/device/listDeviceDetailsByIds) to obtain the device list. The platform returns data including the device serial number and playInfo (which needs to be decrypted; refer to the documentation in 4.1.2.1). Developers need to pay attention to these two data points, as they will be required to enable intercom.

Using the playTalk() interface of the LCOpenSDK_AudioTalk class in the SDK can enable voice intercom. When enabling intercom, the deviceId and playInfo values in the TalkParams intercom parameter class have already been obtained when obtaining the device list; the channelId value is obtained from the message pushed by the platform. For other details, refer to section 4.1.2.6.

4.1.2.10 Enable P2P

Currently, The SDK includes the following interfaces for enabling P2P functionality:

playRtspReal, playTalk, and playDeviceRecord. To enable P2P, it is necessary to add initialization and pre-hole punching steps to the existing playback process.

1. When the app starts logging in, obtain the P2P service interface (/open-api/api-biz/p2p/info) from the developer platform to get P2P service information, and initialize the P2P service information through the SDK's configureP2PServers interface.
2. When retrieving the device list, it is necessary to save the playInfo information of the devices and start pre-hole punching through the SDK's addDevices interface.
3. When pulling the stream, the enableP2P parameter must be set to true. The SDK will use P2P to start pulling the stream internally; if P2P hole punching fails, the SDK will use MTS to pull the stream.

Once the stream is successfully pulled, the type of the current stream can be obtained through the LCOpenSDK_VideoPlay's getCurrentStreamMode() function. If it is STREAM_MODE_P2P, it indicates that the stream was pulled using P2P.

4.1.2.11 Low-power devices

The SDK provides real-time preview capabilities for low-power devices, which can be directly woken up and streamed even while in sleep mode. During the real-time preview, a callback notification is issued when the device enters sleep mode, allowing for interruption of the sleep state.

The status codes in the `onPlayerResult()` callback begin with 80, followed by a two-digit countdown representing seconds. For instance, 8010 signifies that there are 10 seconds left before the device sleeps. When the device does enter sleep mode, it reports the status code "603038." To interrupt the sleep process before the device reaches sleep mode, you can invoke the `rewake()` interface.

After the Demo retrieves the device list, any device in sleep mode will display "sleep" in the list item. Clicking on real-time preview launches streaming for the sleeping device, which will be automatically awakened during the streaming process.

Following a period of real-time preview, the device prepares to enter sleep mode. At this moment, the Demo monitors the status code reported by the SDK and shows the countdown in seconds on the real-time preview page. If the preview page is clicked, the Demo calls the interface to interrupt sleep and continues playback. If no further action is taken, streaming will cease and exit once the countdown concludes.

4.2 Demo Features

4.2.1 Background

APC Demo application is developed using the PC SDK for reference purpose only. It helps developers quickly grasp how to integrate with the Developer Platform PC SDK and gain a comprehensive understanding of its functions.

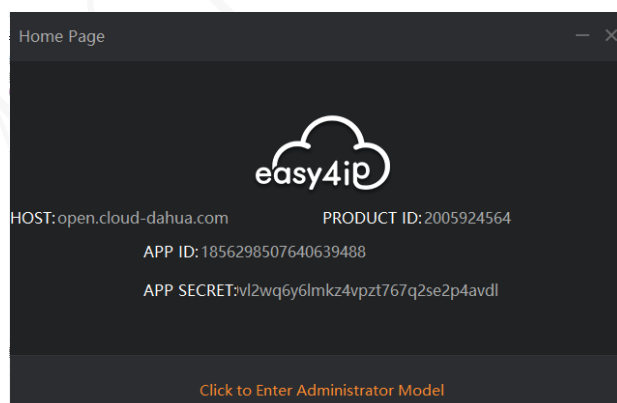
The source code for the PC Demo project can be found in the CPPDemo compressed package. The project primarily references the PC SDK and implements core video-based functions centered around the SDK. You can leverage the provided demo to develop your own client application.

4.2.2 Function Description and Instructions

If you want to understand the primary functions implemented by the provided demo, you can read this document to understand the features of the demo. It is recommended that developers deploy the demo project to their own development environment IDE, ensure it compiles successfully, and then proceed with real-device debugging. By combining this process with the guidance in this document, developers can experience the functions within the demo.

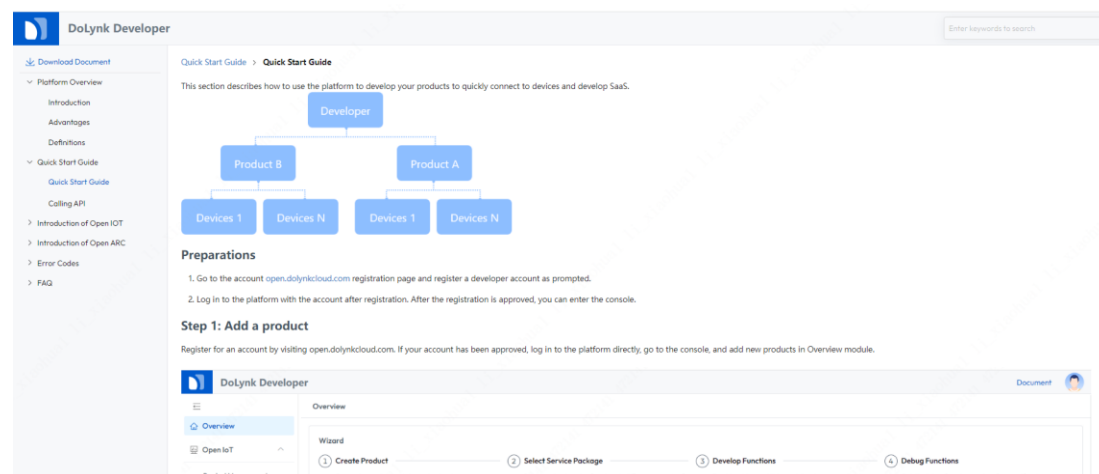
4.2.2.1 Account Login

Double-click Thermallmagery.exe to run the tool.



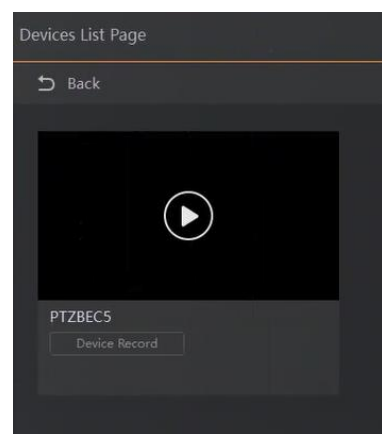
HOST refers to the environment domain name, while the App ID, App Secret, and ProductId can be queried in the product overview section of the developer platform. The product creation has

been completed on the platform at <https://open.dolynkcloud.com/>.



Click **Login** after filling the information to log in to the device list page.

4.2.2.2 Obtaining Device List

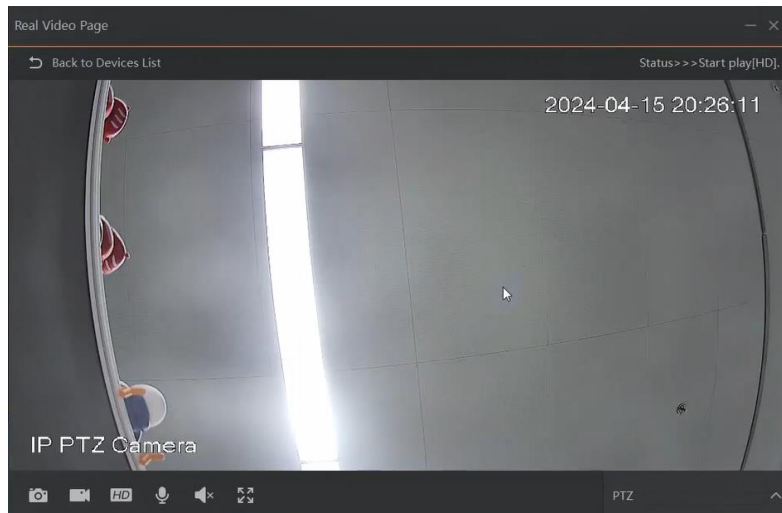


After going to the device list, retrieve the list of devices under the current account from the open platform.

4.2.2.3 Live View and Voice Intercom

On the home page, click the live view button of the camera, and the following window will appear, where you can play the video from the camera.

The windows displays local capture, recording, switching the definition, turning on or off the voice intercom and audio, full-screen switch and PTZ control.



4.2.2.4 PTZ Control

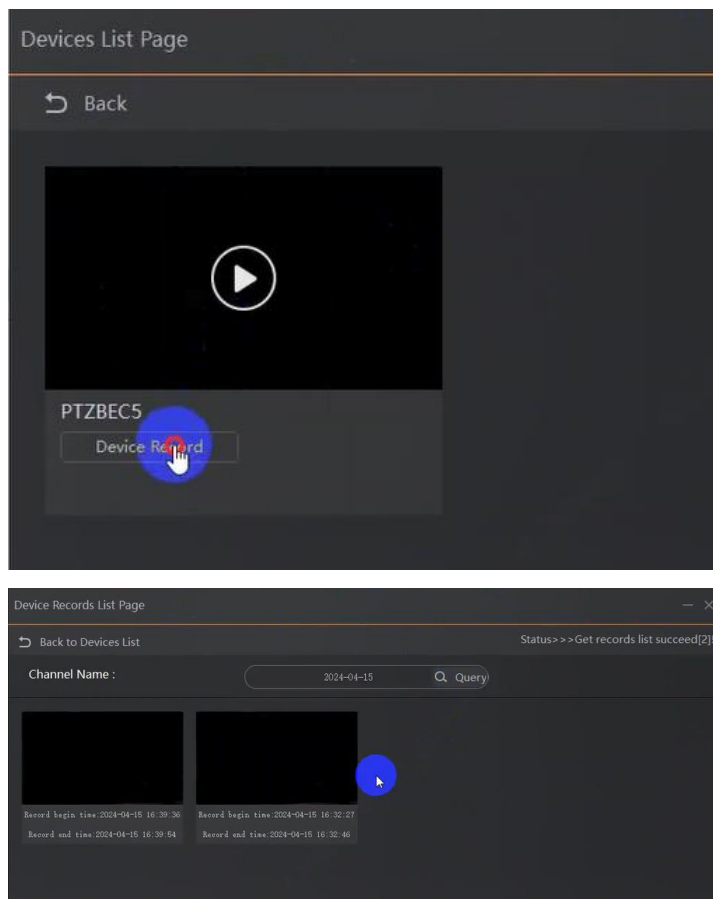


On video playing page, click PTZ to expand the page. Choose the direction to control the camera to make the PTZ rotate up, down, left, or right.

Note: The PTZ function is only supported on PTZ cameras.

4.2.2.5 Local Video Playback

Click **Device Recording** to go to the following screen.

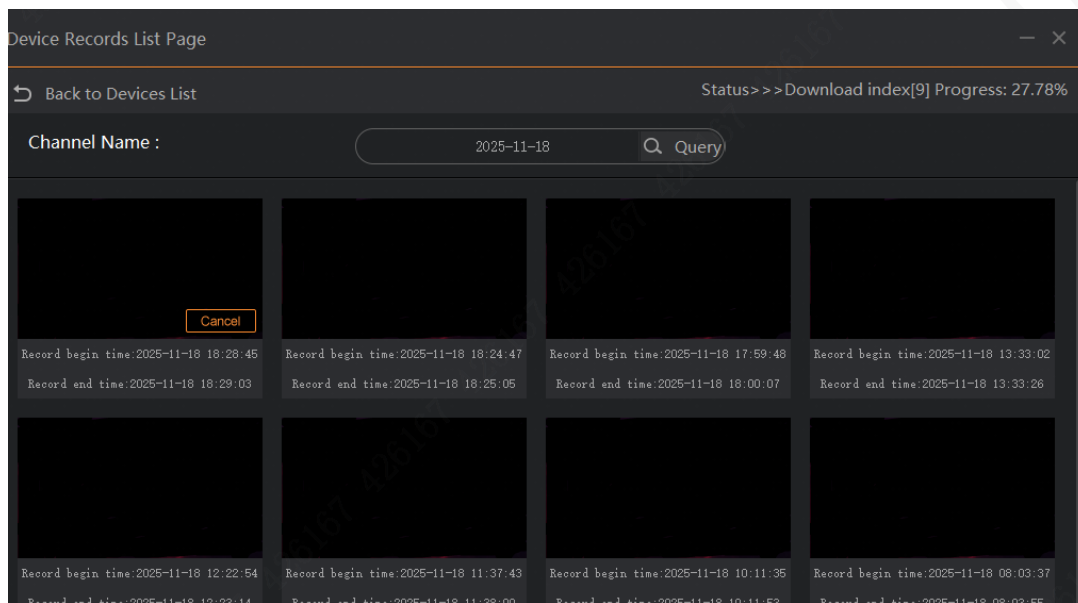
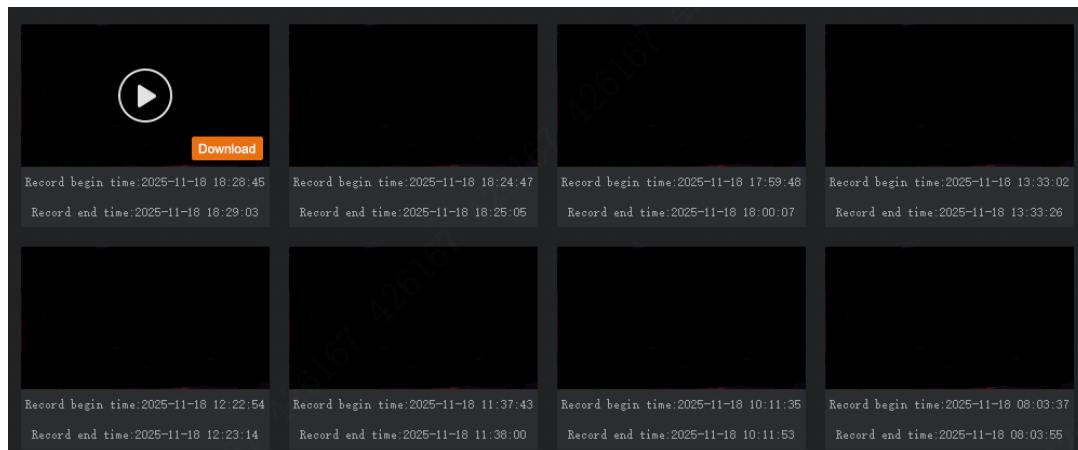


Note: In the demo, by default, only the most recent 10 recordings within the last 24 hours can be queried. Developers can modify the query conditions according to the specific business requirements of their applications.

Click anywhere on recording to go to the video playback screen.

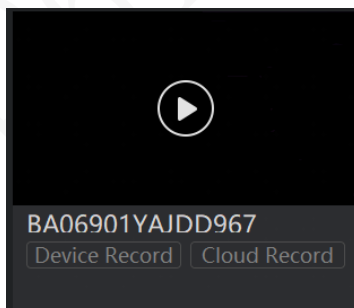
4.2.2.6 Device Record Download

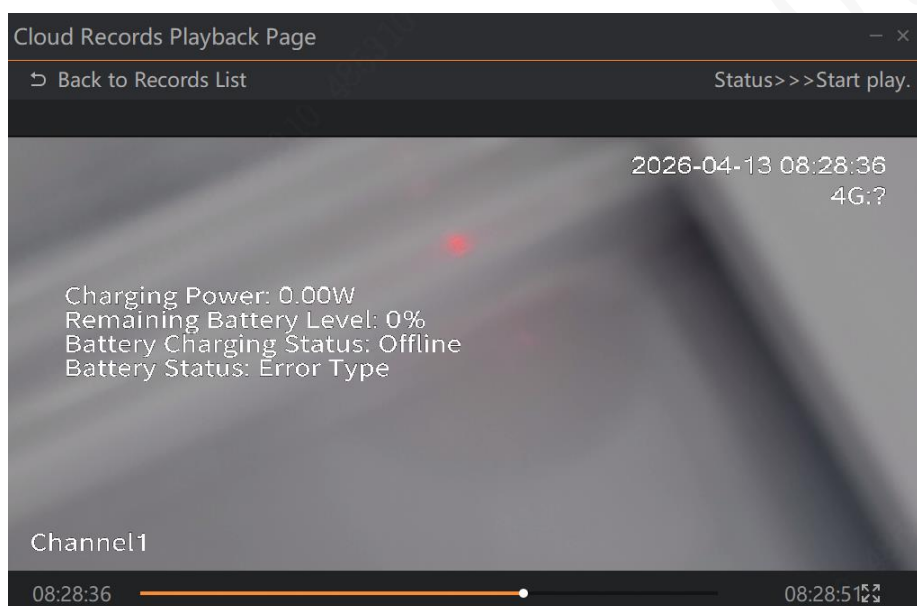
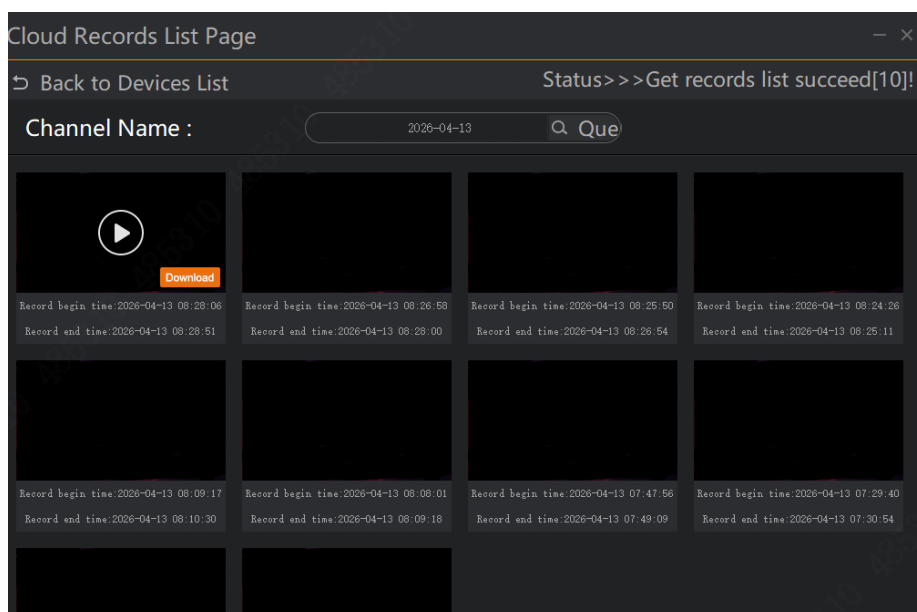
On the device recording interface, hovering the mouse over a file will display "Download," and clicking it will initiate the download. During the download process, the progress can be viewed in the upper right corner (Note: Once a file is downloaded, it cannot be downloaded again. To enable repeated downloads, the upper layer can ensure the uniqueness of the downloaded file name during its own implementation).



4.2.2.7 Cloud Record Playback

Click "Cloud Record" on the recording interface to enter the cloud recording interface. Click the play button in the middle of the file to play back the cloud recording.





4.2.2.8 Cloud Record Download

On the cloud recording interface, hovering the mouse over a file will display "Download," which can be clicked to download. The download progress is displayed in the upper right corner during the download process.

